

Du_Isabelle_HW2

Name: Isabelle Du

Github Username: yxd-1213

USC ID: 5984907579

1. Combined Cycle Power Plant Data Set

(a) Download Data

Package imports

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import statsmodels.api as sm
import statsmodels.formula.api as smf

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_squared_error
```

Get the Cycle Power Plant Data Set

```
In [2]: data = pd.read_excel(
    "CCPP/Folds5x2_pp.xlsx",
    sheet_name="Sheet1"
)

data.head()
```

```
Out[2]:
```

	AT	V	AP	RH	PE
0	14.96	41.76	1024.07	73.17	463.26
1	25.18	62.96	1020.04	59.08	444.37
2	5.11	39.40	1012.16	92.14	488.56
3	20.86	57.32	1010.24	76.64	446.48
4	10.82	37.50	1009.23	96.62	473.90

(b) Exploring the data

i. rows and columns

```
In [4]: # Number of rows and columns  
data.shape
```

```
Out[4]: (9568, 5)
```

```
In [6]: # Column names  
data.columns
```

```
Out[6]: Index(['AT', 'V', 'AP', 'RH', 'PE'], dtype='str')
```

There are 9,568 rows and 5 columns in the dataset.

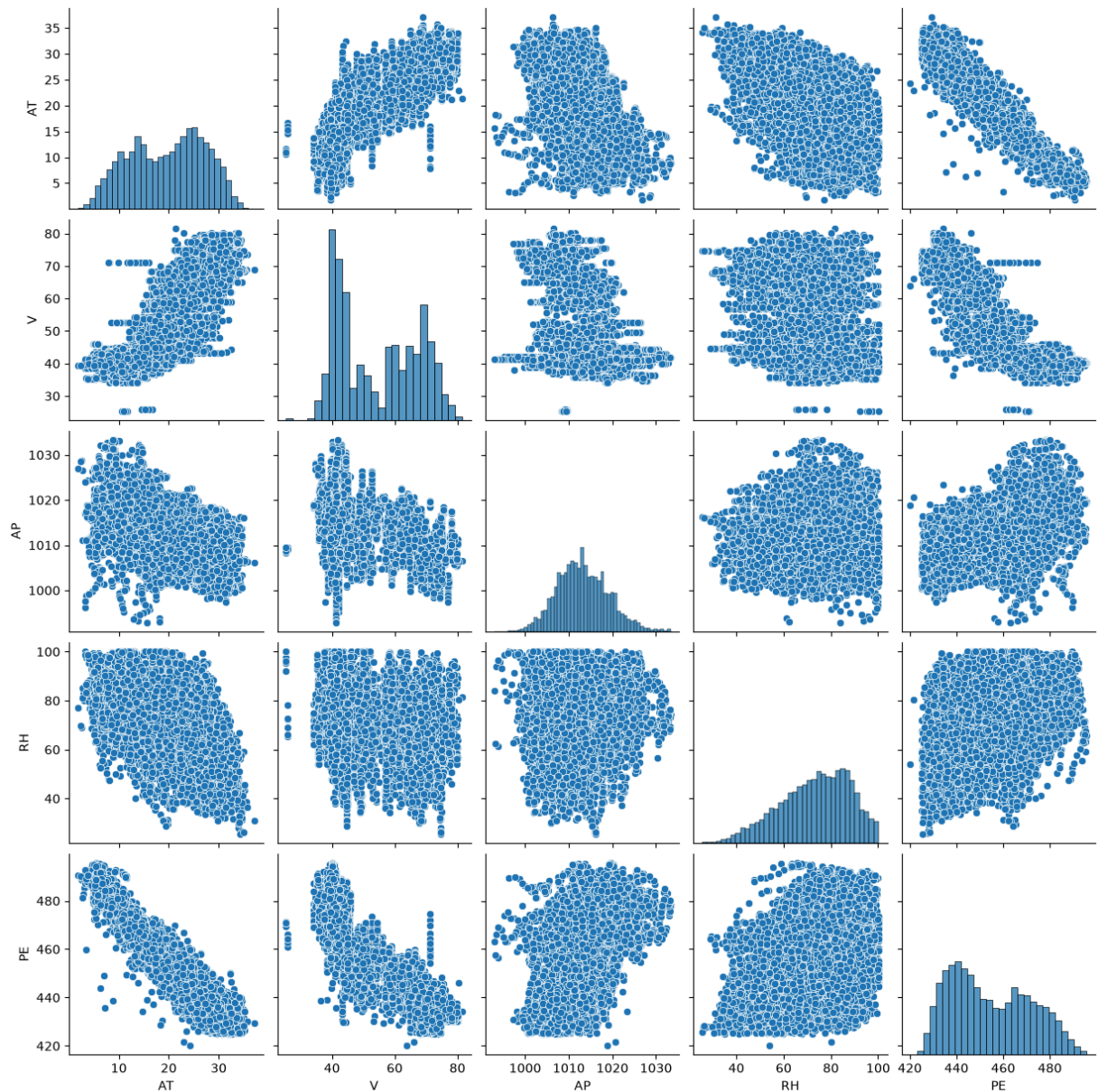
Each row represents one hourly observation collected from the Combined Cycle Power Plant.

The columns represent:

- AT: Ambient Temperature
- V: Exhaust Vacuum
- AP: Ambient Pressure
- RH: Relative Humidity
- PE: Net hourly electrical energy output

ii. pairwise scatterplots of all the variables

```
In [7]: # Pairwise scatterplots of all variables  
sns.pairplot(data)  
plt.show()
```



Findings:

The pairwise scatterplots show that PE has a strong negative relationship with AT and V. As ambient temperature and exhaust vacuum increase, the electrical power output generally decreases.

AP appears to have a weaker positive relationship with PE, while RH also shows a relatively weak positive relationship with PE.

There are also noticeable relationships among some of the predictors. For example, AT and V are positively associated, while AT appears to be negatively associated with AP and RH. Overall, AT and V seem to have the strongest relationships with PE.

iii. mean, the median, range, first and third quartiles, and interquartile ranges

In [8]: *# Summary statistics for each variable*

```
summary_stats = pd.DataFrame({
    'Mean': data.mean(),
    'Median': data.median(),
    'Range': data.max() - data.min(),
    'Q1': data.quantile(0.25),
    'Q3': data.quantile(0.75),
    'IQR': data.quantile(0.75) - data.quantile(0.25)
})

summary_stats
```

Out[8]:

	Mean	Median	Range	Q1	Q3	IQR
AT	19.651231	20.345	35.30	13.5100	25.72	12.2100
V	54.305804	52.080	56.20	41.7400	66.54	24.8000
AP	1013.259078	1012.940	40.41	1009.1000	1017.26	8.1600
RH	73.308978	74.975	74.60	63.3275	84.83	21.5025
PE	454.365009	451.550	75.50	439.7500	468.43	28.6800

(c) Simple Linear Regression

In [9]: *# Simple linear regression for each predictor*

```
predictors = ['AT', 'V', 'AP', 'RH']
simple_models = {}

for predictor in predictors:
    model = smf.ols(f'PE ~ {predictor}', data=data).fit()
    simple_models[predictor] = model
```

In [10]: *# Summary of simple linear regression results*

```
simple_results = []

for predictor in predictors:
    model = simple_models[predictor]

    simple_results.append({
        'Predictor': predictor,
        'Coefficient': model.params[predictor],
        'Std. Error': model.bse[predictor],
        't-statistic': model.tvalues[predictor],
        'p-value': model.pvalues[predictor],
        'R-squared': model.rsquared
    })

simple_results = pd.DataFrame(simple_results)
simple_results
```

Out [10]:

	Predictor	Coefficient	Std. Error	t-statistic	p-value	R-squared
0	AT	-2.171320	0.007443	-291.715195	0.0	0.898948
1	V	-1.168135	0.006776	-172.401540	0.0	0.756518
2	AP	1.489872	0.025126	59.296232	0.0	0.268769
3	RH	0.455650	0.011006	41.398730	0.0	0.151939

Results:

All four predictors have statistically significant associations with PE, since their p-values are all below 0.05.

AT has a strong negative association with PE, with a coefficient of -2.1713 and an R-squared of 0.8989. This indicates that AT alone explains about 89.9% of the variation in PE.

V also has a strong negative association with PE, with a coefficient of -1.1681 and an R-squared of 0.7565.

AP and RH both have positive and statistically significant associations with PE, but their R-squared values are much lower (0.2688 and 0.1519, respectively), indicating weaker explanatory power when used individually.

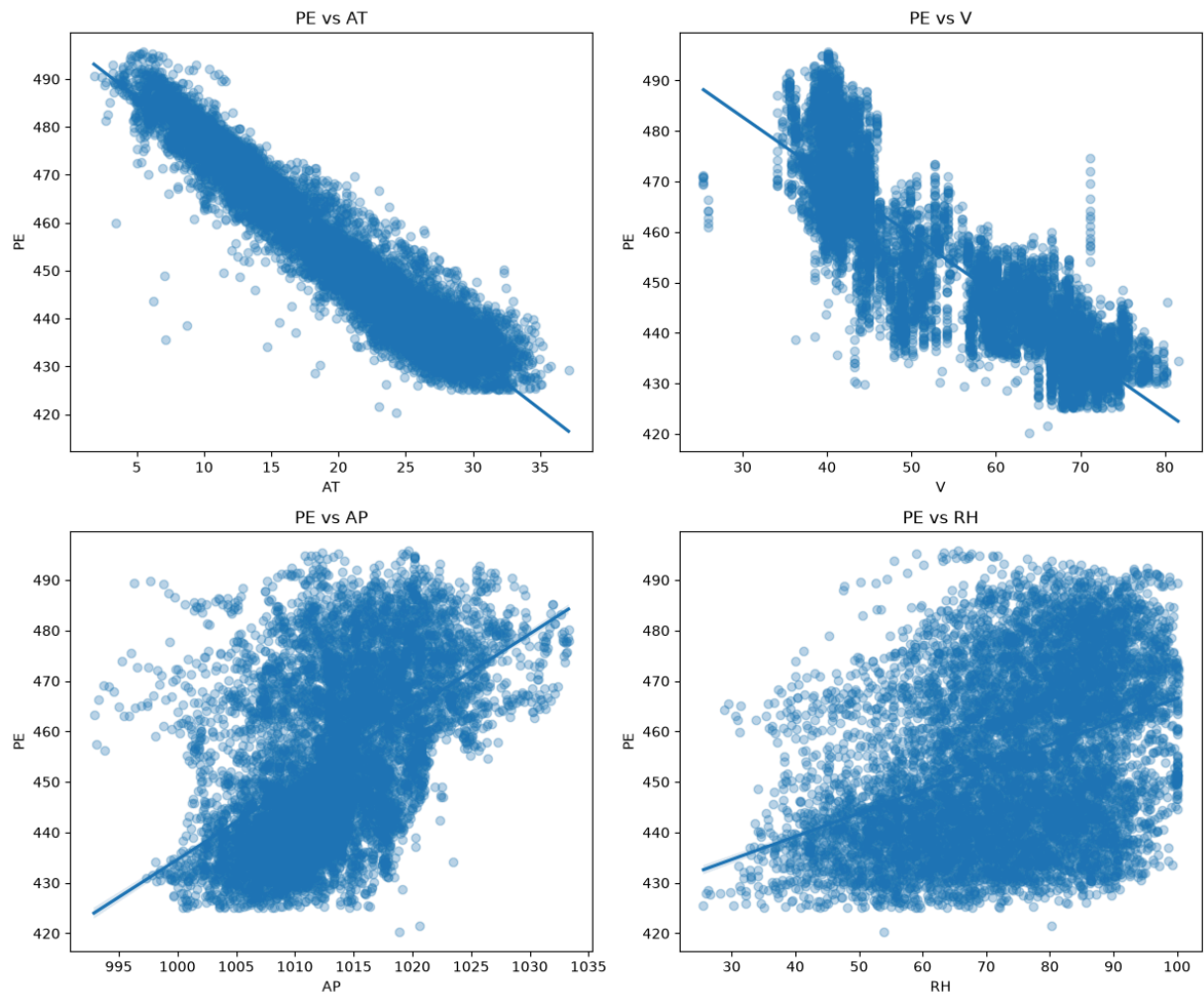
Overall, AT shows the strongest simple linear relationship with PE, followed by V.

```
In [11]: # Scatterplots with simple linear regression lines

fig, axes = plt.subplots(2, 2, figsize=(12, 10))

for predictor, ax in zip(predictors, axes.flatten()):
    sns.regplot(x=predictor, y='PE', data=data,
                scatter_kws={'alpha': 0.3}, ax=ax)
    ax.set_title(f'PE vs {predictor}')

plt.tight_layout()
plt.show()
```



```
In [12]: # Check potential outliers using studentized residuals

outlier_results = []

for predictor in predictors:
    model = simple_models[predictor]
    influence = model.get_influence()
    studentized_residuals = influence.resid_studentized_internal

    outliers = np.where(np.abs(studentized_residuals) > 3)[0]

    outlier_results.append({
        'Predictor': predictor,
        'Number of Potential Outliers': len(outliers)
    })

outlier_results = pd.DataFrame(outlier_results)
outlier_results
```

Out [12]:

	Predictor	Number of Potential Outliers
0	AT	42
1	V	33
2	AP	29
3	RH	2

	Predictor	Number of Potential Outliers
0	AT	42
1	V	33
2	AP	29
3	RH	2

```
In [13]: # Studentized residual plots

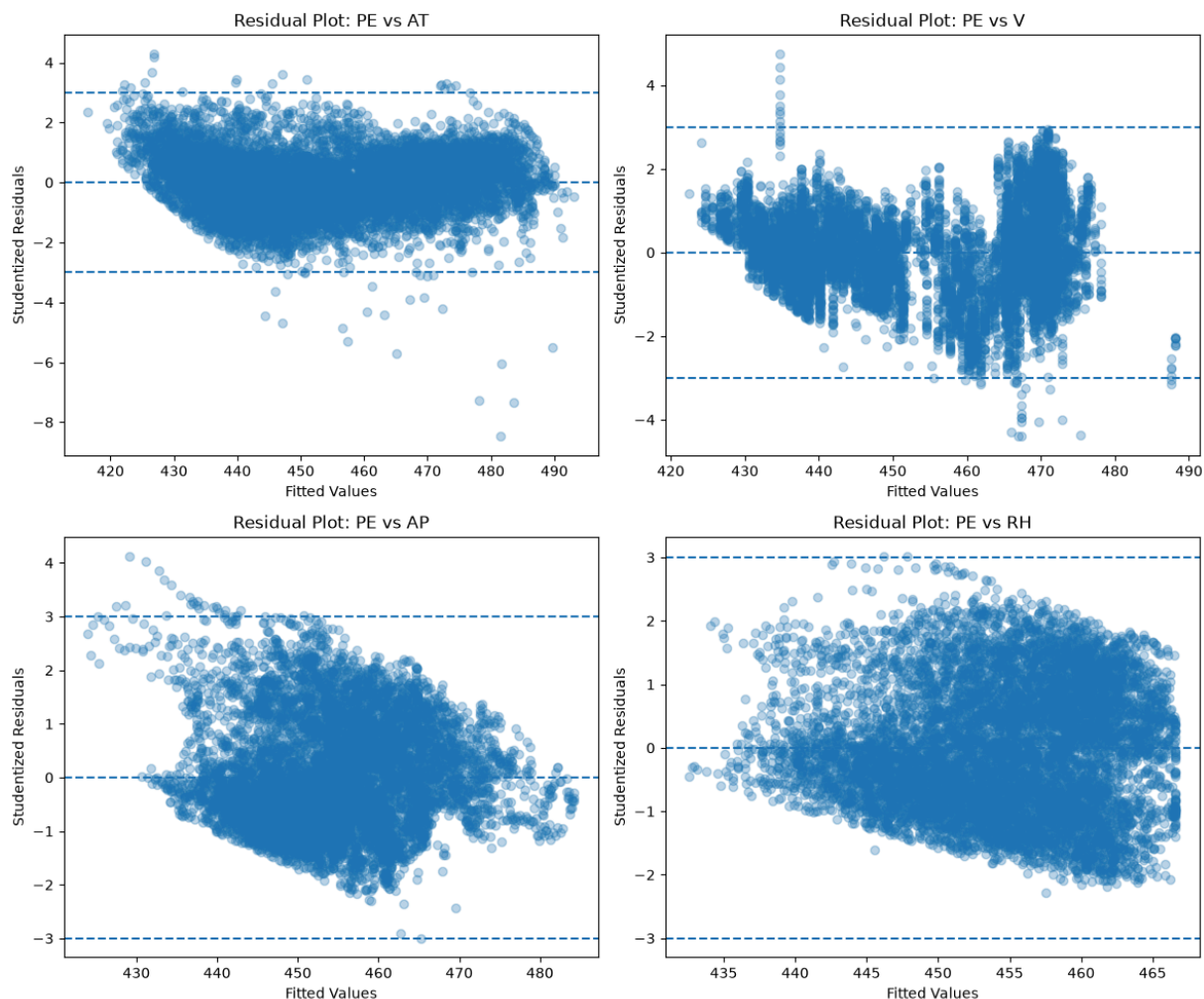
fig, axes = plt.subplots(2, 2, figsize=(12, 10))

for predictor, ax in zip(predictors, axes.flatten()):
    model = simple_models[predictor]
    influence = model.get_influence()
    studentized_residuals = influence.resid_studentized_internal

    ax.scatter(model.fittedvalues, studentized_residuals, alpha=0.3)
    ax.axhline(3, linestyle='--')
    ax.axhline(-3, linestyle='--')
    ax.axhline(0, linestyle='--')

    ax.set_title(f'Residual Plot: PE vs {predictor}')
    ax.set_xlabel('Fitted Values')
    ax.set_ylabel('Studentized Residuals')

plt.tight_layout()
plt.show()
```



Outliers:

Using studentized residuals with a threshold of $|3|$, there are 42 potential outliers for AT, 33 for V, 29 for AP, and 2 for RH.

Although some observations have large studentized residuals, I would not remove them without evidence that they are measurement or data-entry errors. The residual plots also show some systematic patterns, especially for V, AP, and RH, suggesting that part of the issue may come from nonlinearity or model misspecification rather than true outliers.

(d) Multiple Regression

```
In [14]: # Multiple linear regression using all predictors

multiple_model = smf.ols(
    'PE ~ AT + V + AP + RH',
    data=data
).fit()

multiple_model.summary()
```

Out [14]:

OLS Regression Results

Dep. Variable:	PE	R-squared:	0.929
Model:	OLS	Adj. R-squared:	0.929
Method:	Least Squares	F-statistic:	3.114e+04
Date:	Fri, 25 Sep 2026	Prob (F-statistic):	0.00
Time:	14:41:50	Log-Likelihood:	-28088.
No. Observations:	9568	AIC:	5.619e+04
Df Residuals:	9563	BIC:	5.622e+04
Df Model:	4		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	454.6093	9.749	46.634	0.000	435.500	473.718
AT	-1.9775	0.015	-129.342	0.000	-2.007	-1.948
V	-0.2339	0.007	-32.122	0.000	-0.248	-0.220
AP	0.0621	0.009	6.564	0.000	0.044	0.081
RH	-0.1581	0.004	-37.918	0.000	-0.166	-0.150

Omnibus:	892.002	Durbin-Watson:	2.033
Prob(Omnibus):	0.000	Jarque-Bera (JB):	4086.777
Skew:	-0.352	Prob(JB):	0.00
Kurtosis:	6.123	Cond. No.	2.13e+05

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 2.13e+05. This might indicate that there are strong multicollinearity or other numerical problems.

In [15]: *# Coefficients and p-values of the multiple regression model*

```
multiple_results = pd.DataFrame({
    'Coefficient': multiple_model.params,
    'Std. Error': multiple_model.bse,
    't-statistic': multiple_model.tvalues,
    'p-value': multiple_model.pvalues
})

multiple_results
```

Out [15]:

	Coefficient	Std. Error	t-statistic	p-value
Intercept	454.609274	9.748512	46.633709	0.000000e+00
AT	-1.977513	0.015289	-129.342024	0.000000e+00
V	-0.233916	0.007282	-32.122109	4.375305e-215
AP	0.062083	0.009458	6.564086	5.507109e-11
RH	-0.158054	0.004168	-37.918473	3.104584e-293

Results:

The multiple linear regression model has an R-squared of 0.929, indicating that approximately 92.9% of the variation in PE is explained by AT, V, AP, and RH together.

All four predictors are statistically significant because their p-values are below 0.05. Therefore, we reject the null hypothesis $H_0: \beta_j = 0$ for AT, V, AP, and RH.

Holding the other predictors fixed:

- A one-unit increase in AT is associated with about a 1.9775-unit decrease in PE.
- A one-unit increase in V is associated with about a 0.2339-unit decrease in PE.
- A one-unit increase in AP is associated with about a 0.0621-unit increase in PE.
- A one-unit increase in RH is associated with about a 0.1581-unit decrease in PE.

(e) 1c Compare to 1d

```
In [16]: # Compare simple and multiple regression coefficients

coefficient_comparison = pd.DataFrame({
    'Simple Regression': simple_results.set_index('Predictor')['Coefficient'],
    'Multiple Regression': multiple_model.params[predictors]
})

coefficient_comparison
```

Out [16]:

	Simple Regression	Multiple Regression
AT	-2.171320	-1.977513
V	-1.168135	-0.233916
AP	1.489872	0.062083
RH	0.455650	-0.158054

```
In [17]: # Plot simple vs multiple regression coefficients

plt.figure(figsize=(7, 6))

plt.scatter(
```

```

coefficient_comparison['Simple Regression'],
coefficient_comparison['Multiple Regression']
)

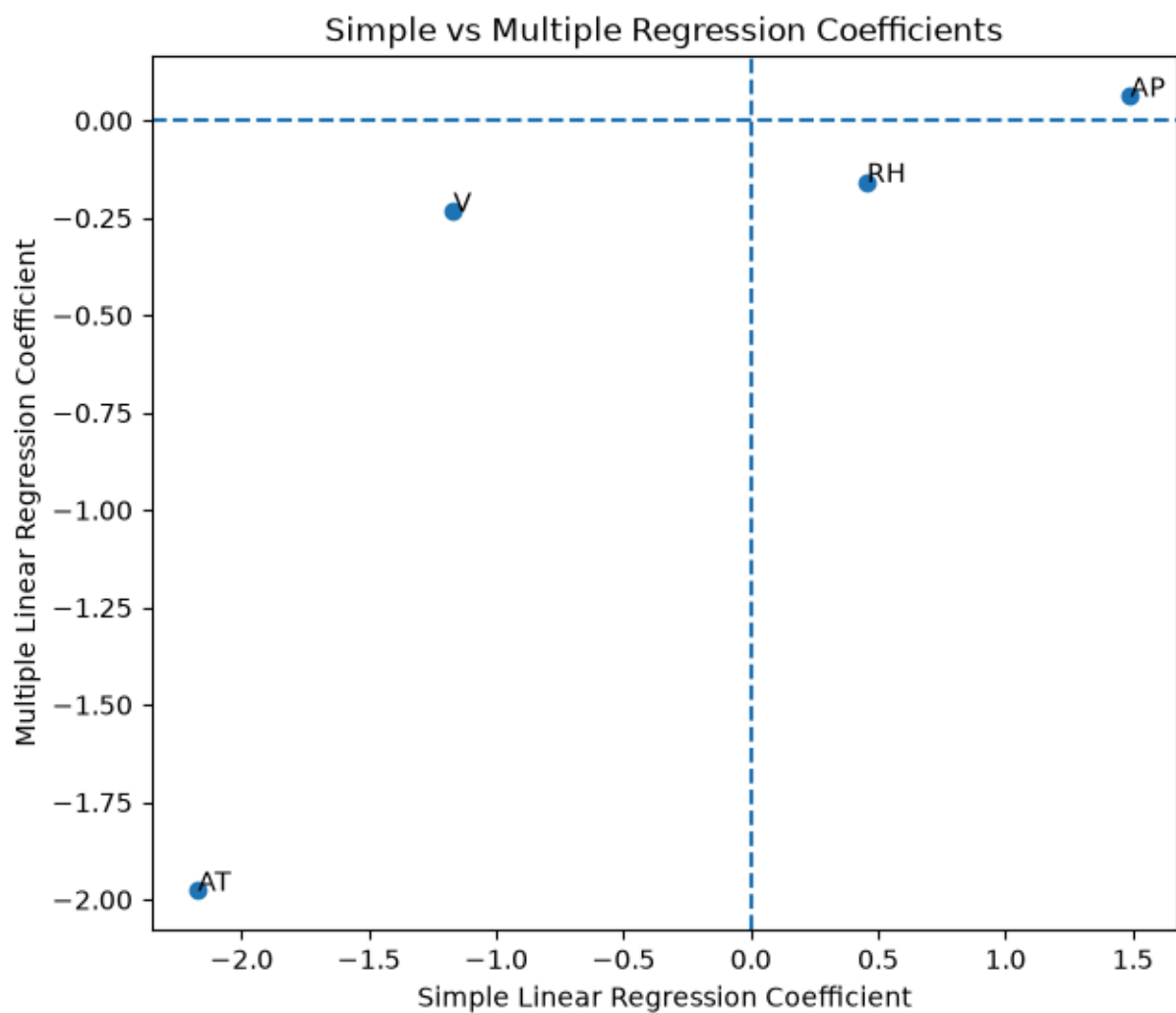
for predictor in predictors:
    plt.annotate(
        predictor,
        (
            coefficient_comparison.loc[predictor, 'Simple Regression'],
            coefficient_comparison.loc[predictor, 'Multiple Regression']
        )
    )

plt.axhline(0, linestyle='--')
plt.axvline(0, linestyle='--')

plt.xlabel('Simple Linear Regression Coefficient')
plt.ylabel('Multiple Linear Regression Coefficient')
plt.title('Simple vs Multiple Regression Coefficients')

plt.show()

```



Comparison:

The coefficients from the simple and multiple regression models are noticeably different.

AT changes only slightly, from -2.1713 in the simple regression to -1.9775 in the multiple regression.

The coefficient of V decreases substantially in magnitude, from -1.1681 to -0.2339. AP also decreases greatly, from 1.4899 to 0.0621.

The largest qualitative change occurs for RH. Its coefficient is positive in the simple regression (0.4557) but becomes negative in the multiple regression (-0.1581).

These differences occur because the simple regression considers each predictor separately, while the multiple regression estimates the association of each predictor with PE while holding the other predictors fixed. Since some predictors are related to each other, their estimated coefficients can change when they are included together in the model.

(f) Nonlinear Association

```
In [18]: # Cubic polynomial regression for each predictor

polynomial_models = {}

for predictor in predictors:
    model = smf.ols(
        f'PE ~ {predictor} + I({predictor}**2) + I({predictor}**3)',
        data=data
    ).fit()

    polynomial_models[predictor] = model
```

```
In [19]: # P-values for quadratic and cubic terms

nonlinear_results = []

for predictor in predictors:
    model = polynomial_models[predictor]

    nonlinear_results.append({
        'Predictor': predictor,
        'Linear p-value': model.pvalues[predictor],
        'Quadratic p-value': model.pvalues[f'I({predictor} ** 2)'],
        'Cubic p-value': model.pvalues[f'I({predictor} ** 3)'],
        'R-squared': model.rsquared
    })

nonlinear_results = pd.DataFrame(nonlinear_results)
nonlinear_results
```

Out [19]:

	Predictor	Linear p-value	Quadratic p-value	Cubic p-value	R-squared
0	AT	7.898147e-07	8.833045e-73	3.652185e-110	0.911883
1	V	2.526589e-05	7.684969e-01	1.373489e-02	0.775022
2	AP	4.502735e-17	3.666705e-17	8.264146e-18	0.274863
3	RH	3.772510e-04	9.395430e-06	1.440279e-05	0.153743

Results:

There is evidence of nonlinear association between PE and all four predictors.

For AT, both the quadratic and cubic terms are statistically significant. For V, the quadratic term is not significant, but the cubic term is significant, which still provides evidence of a nonlinear relationship. For AP and RH, both the quadratic and cubic terms are statistically significant.

Therefore, AT, V, AP, and RH all show evidence of nonlinear association with PE.

(g) Interactions of PredictorsIn [20]: *# Full linear regression model with all pairwise interactions*

```
interaction_model = smf.ols(
    'PE ~ (AT + V + AP + RH) ** 2',
    data=data
).fit()

interaction_model.summary()
```

Out [20] :

OLS Regression Results

Dep. Variable:	PE	R-squared:	0.936
Model:	OLS	Adj. R-squared:	0.936
Method:	Least Squares	F-statistic:	1.405e+04
Date:	Fri, 25 Sep 2026	Prob (F-statistic):	0.00
Time:	14:55:32	Log-Likelihood:	-27548.
No. Observations:	9568	AIC:	5.512e+04
Df Residuals:	9557	BIC:	5.520e+04
Df Model:	10		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
Intercept	685.7825	78.640	8.721	0.000	531.631	839.934
AT	-4.3470	2.373	-1.832	0.067	-8.999	0.305
V	-7.6749	1.351	-5.682	0.000	-10.323	-5.027
AP	-0.1524	0.077	-1.983	0.047	-0.303	-0.002
RH	1.5709	0.773	2.031	0.042	0.055	3.087
AT:V	0.0210	0.001	23.338	0.000	0.019	0.023
AT:AP	0.0018	0.002	0.752	0.452	-0.003	0.006
AT:RH	-0.0052	0.001	-6.444	0.000	-0.007	-0.004
V:AP	0.0068	0.001	5.135	0.000	0.004	0.009
V:RH	0.0008	0.000	1.716	0.086	-0.000	0.002
AP:RH	-0.0016	0.001	-2.125	0.034	-0.003	-0.000

Omnibus:	1454.609	Durbin-Watson:	2.030
Prob(Omnibus):	0.000	Jarque-Bera (JB):	9170.848
Skew:	-0.574	Prob(JB):	0.00
Kurtosis:	7.657	Cond. No.	1.70e+08

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1.7e+08. This might indicate that there are strong multicollinearity or other numerical problems.

```
In [21]: # P-values of pairwise interaction terms

interaction_terms = [
    'AT:V',
    'AT:AP',
    'AT:RH',
    'V:AP',
    'V:RH',
    'AP:RH'
]

interaction_results = pd.DataFrame({
    'Coefficient': interaction_model.params[interaction_terms],
    'p-value': interaction_model.pvalues[interaction_terms]
})

interaction_results
```

```
Out[21]:
```

	Coefficient	p-value
AT:V	0.020971	3.333358e-117
AT:AP	0.001759	4.520509e-01
AT:RH	-0.005230	1.216944e-10
V:AP	0.006812	2.877026e-07
V:RH	0.000839	8.619366e-02
AP:RH	-0.001612	3.360557e-02

Results:

Four of the six pairwise interaction terms are statistically significant at the 0.05 level: AT:V, AT:RH, V:AP, and AP:RH.

The interaction terms AT:AP and V:RH are not statistically significant because their p-values are greater than 0.05.

Therefore, there is evidence that some predictors interact with each other in their association with PE.

(h) Improvement

```
In [22]: # Split data into 70% training and 30% testing sets

train_data, test_data = train_test_split(
    data,
    test_size=0.30,
    random_state=1
)
```

```
print("Training set size:", train_data.shape)
print("Testing set size:", test_data.shape)
```

Training set size: (6697, 5)

Testing set size: (2871, 5)

In [23]: *# Multiple linear regression with all predictors*

```
linear_train_model = smf.ols(
    'PE ~ AT + V + AP + RH',
    data=train_data
).fit()

# Predictions
linear_train_pred = linear_train_model.predict(train_data)
linear_test_pred = linear_train_model.predict(test_data)

# MSE
linear_train_mse = mean_squared_error(
    train_data['PE'],
    linear_train_pred
)

linear_test_mse = mean_squared_error(
    test_data['PE'],
    linear_test_pred
)

print("Linear Regression Train MSE:", linear_train_mse)
print("Linear Regression Test MSE:", linear_test_mse)
```

Linear Regression Train MSE: 20.766119761450952

Linear Regression Test MSE: 20.77747810688431

In [24]: *# Full model with quadratic terms and pairwise interactions*

```
full_formula = (
    'PE ~ AT + V + AP + RH '
    '+ I(AT**2) + I(V**2) + I(AP**2) + I(RH**2) '
    '+ AT:V + AT:AP + AT:RH + V:AP + V:RH + AP:RH'
)

full_improved_model = smf.ols(
    full_formula,
    data=train_data
).fit()

full_improved_model.summary()
```

Out [24] :

OLS Regression Results

Dep. Variable:		PE		R-squared:		0.938
Model:		OLS		Adj. R-squared:		0.938
Method:		Least Squares		F-statistic:		7181.
Date:		Fri, 25 Sep 2026		Prob (F-statistic):		0.00
Time:		14:59:01		Log-Likelihood:		-19192.
No. Observations:		6697		AIC:		3.841e+04
Df Residuals:		6682		BIC:		3.852e+04
Df Model:		14				
Covariance Type:		nonrobust				
		coef	std err	t	P> t	[0.025 0.975]
Intercept		-8171.8080	1416.708	-5.768	0.000	-1.09e+04 -5394.608
AT		-3.8103	3.684	-1.034	0.301	-11.032 3.412
V		-3.7163	1.786	-2.081	0.037	-7.217 -0.216
AP		16.8844	2.745	6.151	0.000	11.504 22.265
RH		4.7922	1.060	4.521	0.000	2.714 6.870
I(AT ** 2)		0.0159	0.004	4.341	0.000	0.009 0.023
I(V ** 2)		-0.0009	0.001	-1.010	0.312	-0.003 0.001
I(AP ** 2)		-0.0082	0.001	-6.165	0.000	-0.011 -0.006
I(RH ** 2)		-0.0020	0.000	-6.346	0.000	-0.003 -0.001
AT:V		0.0117	0.003	3.692	0.000	0.005 0.018
AT:AP		0.0012	0.004	0.344	0.731	-0.006 0.008
AT:RH		-0.0067	0.002	-3.664	0.000	-0.010 -0.003
V:AP		0.0032	0.002	1.844	0.065	-0.000 0.007
V:RH		0.0005	0.001	0.620	0.535	-0.001 0.002
AP:RH		-0.0045	0.001	-4.352	0.000	-0.006 -0.002
Omnibus:		1461.187	Durbin-Watson:		2.028	
Prob(Omnibus):		0.000	Jarque-Bera (JB):		12922.950	
Skew:		-0.793	Prob(JB):		0.00	
Kurtosis:		9.618	Cond. No.		2.81e+10	

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The condition number is large, 2.81e+10. This might indicate that there are strong multicollinearity or other numerical problems.

In [25]: *# Reduced model after removing insignificant higher-order terms*

```
reduced_formula = (  
    'PE ~ AT + V + AP + RH '  
    '+ I(AT**2) + I(AP**2) + I(RH**2) '  
    '+ AT:V + AT:RH + AP:RH'  
)  
  
reduced_model = smf.ols(  
    reduced_formula,  
    data=train_data  
) .fit()  
  
reduced_model.summary()
```

Out [25] :

OLS Regression Results

Dep. Variable:		PE	R-squared:		0.938	
Model:		OLS	Adj. R-squared:		0.937	
Method:		Least Squares	F-statistic:		1.004e+04	
Date:		Fri, 25 Sep 2026	Prob (F-statistic):		0.00	
Time:		15:01:04	Log-Likelihood:		-19198.	
No. Observations:		6697	AIC:		3.842e+04	
Df Residuals:		6686	BIC:		3.849e+04	
Df Model:		10				
Covariance Type:		nonrobust				
	coef	std err	t	P> t	[0.025	0.975]
Intercept	-1.052e+04	1079.936	-9.738	0.000	-1.26e+04	-8399.884
AT	-2.3804	0.101	-23.552	0.000	-2.579	-2.182
V	-0.4548	0.031	-14.496	0.000	-0.516	-0.393
AP	21.2291	2.134	9.950	0.000	17.046	25.412
RH	6.0882	0.763	7.976	0.000	4.592	7.585
I(AT ** 2)	0.0174	0.002	7.655	0.000	0.013	0.022
I(AP ** 2)	-0.0102	0.001	-9.687	0.000	-0.012	-0.008
I(RH ** 2)	-0.0021	0.000	-7.575	0.000	-0.003	-0.002
AT:V	0.0076	0.001	5.302	0.000	0.005	0.010
AT:RH	-0.0071	0.001	-8.079	0.000	-0.009	-0.005
AP:RH	-0.0057	0.001	-7.722	0.000	-0.007	-0.004
Omnibus:		1462.479	Durbin-Watson:		2.027	
Prob(Omnibus):		0.000	Jarque-Bera (JB):		12609.274	
Skew:		-0.802	Prob(JB):		0.00	
Kurtosis:		9.528	Cond. No.		2.14e+10	

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The condition number is large, 2.14e+10. This might indicate that there are strong multicollinearity or other numerical problems.

```
In [27]: # Predictions from the reduced improved model

reduced_train_pred = reduced_model.predict(train_data)
reduced_test_pred = reduced_model.predict(test_data)

# Train and test MSE
reduced_train_mse = mean_squared_error(
    train_data['PE'],
    reduced_train_pred
)

reduced_test_mse = mean_squared_error(
    test_data['PE'],
    reduced_test_pred
)

print("Improved Model Train MSE:", reduced_train_mse)
print("Improved Model Test MSE:", reduced_test_mse)
```

Improved Model Train MSE: 18.093235651112646
 Improved Model Test MSE: 18.2635881122737

```
In [28]: # Compare the two models

mse_comparison = pd.DataFrame({
    'Model': ['Linear Regression', 'Improved Regression'],
    'Train MSE': [linear_train_mse, reduced_train_mse],
    'Test MSE': [linear_test_mse, reduced_test_mse]
})

mse_comparison
```

```
Out[28]:
```

	Model	Train MSE	Test MSE
0	Linear Regression	20.766120	20.777478
1	Improved Regression	18.093236	18.263588

Results:

The ordinary multiple linear regression model has a training MSE of 20.7661 and a test MSE of 20.7775.

After including significant quadratic and interaction terms, the improved regression model has a lower training MSE of 18.0932 and a lower test MSE of 18.2636.

Therefore, adding nonlinear and interaction terms improves the predictive performance of the regression model.

(i) KNN

```
In [29]: # Prepare training and testing data for KNN
```

```
X_train = train_data[['AT', 'V', 'AP', 'RH']]
y_train = train_data['PE']

X_test = test_data[['AT', 'V', 'AP', 'RH']]
y_test = test_data['PE']
```

```
In [30]: # KNN regression using raw features

k_values = range(1, 101)

raw_train_mse = []
raw_test_mse = []

for k in k_values:
    knn = KNeighborsRegressor(n_neighbors=k)
    knn.fit(X_train, y_train)

    train_pred = knn.predict(X_train)
    test_pred = knn.predict(X_test)

    raw_train_mse.append(
        mean_squared_error(y_train, train_pred)
    )

    raw_test_mse.append(
        mean_squared_error(y_test, test_pred)
    )
```

```
In [31]: # Best k for raw features

best_raw_index = np.argmin(raw_test_mse)
best_raw_k = list(k_values)[best_raw_index]
best_raw_test_mse = raw_test_mse[best_raw_index]

print("Best k for raw features:", best_raw_k)
print("Best test MSE for raw features:", best_raw_test_mse)
```

Best k for raw features: 5
Best test MSE for raw features: 15.704821203761764

```
In [32]: # Normalize the predictor variables

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
In [33]: # KNN regression using normalized features

normalized_train_mse = []
normalized_test_mse = []

for k in k_values:
    knn = KNeighborsRegressor(n_neighbors=k)
    knn.fit(X_train_scaled, y_train)
```

```
train_pred = knn.predict(X_train_scaled)
test_pred = knn.predict(X_test_scaled)

normalized_train_mse.append(
    mean_squared_error(y_train, train_pred)
)

normalized_test_mse.append(
    mean_squared_error(y_test, test_pred)
)
```

In [34]: *# Best k for normalized features*

```
best_normalized_index = np.argmin(normalized_test_mse)
best_normalized_k = list(k_values)[best_normalized_index]
best_normalized_test_mse = normalized_test_mse[best_normalized_index]

print("Best k for normalized features:", best_normalized_k)
print("Best test MSE for normalized features:", best_normalized_test_mse)
```

Best k for normalized features: 4

Best test MSE for normalized features: 14.070606067136888

In [35]: *# Train and test MSE for raw features versus 1/k*

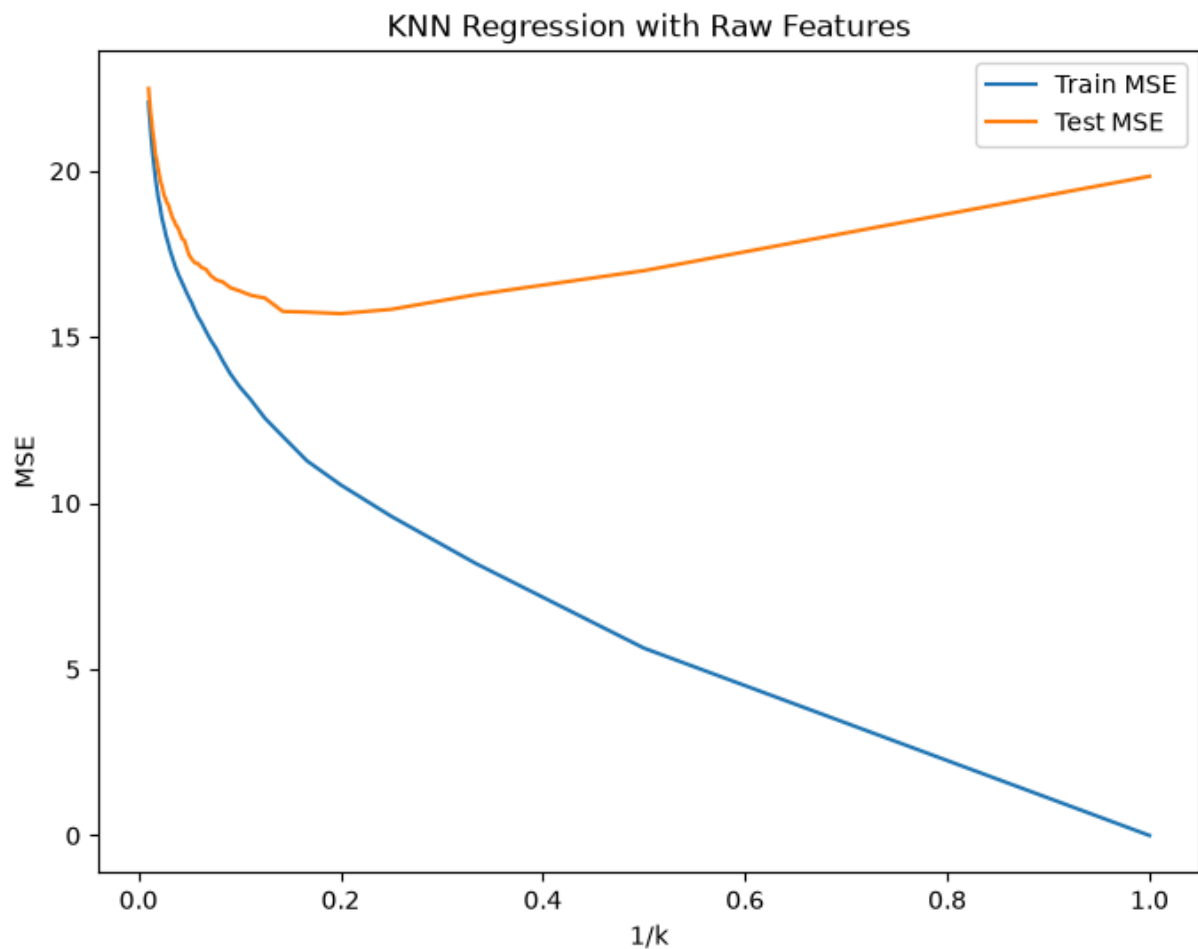
```
inverse_k = 1 / np.array(list(k_values))

plt.figure(figsize=(8, 6))

plt.plot(inverse_k, raw_train_mse, label='Train MSE')
plt.plot(inverse_k, raw_test_mse, label='Test MSE')

plt.xlabel('1/k')
plt.ylabel('MSE')
plt.title('KNN Regression with Raw Features')
plt.legend()

plt.show()
```



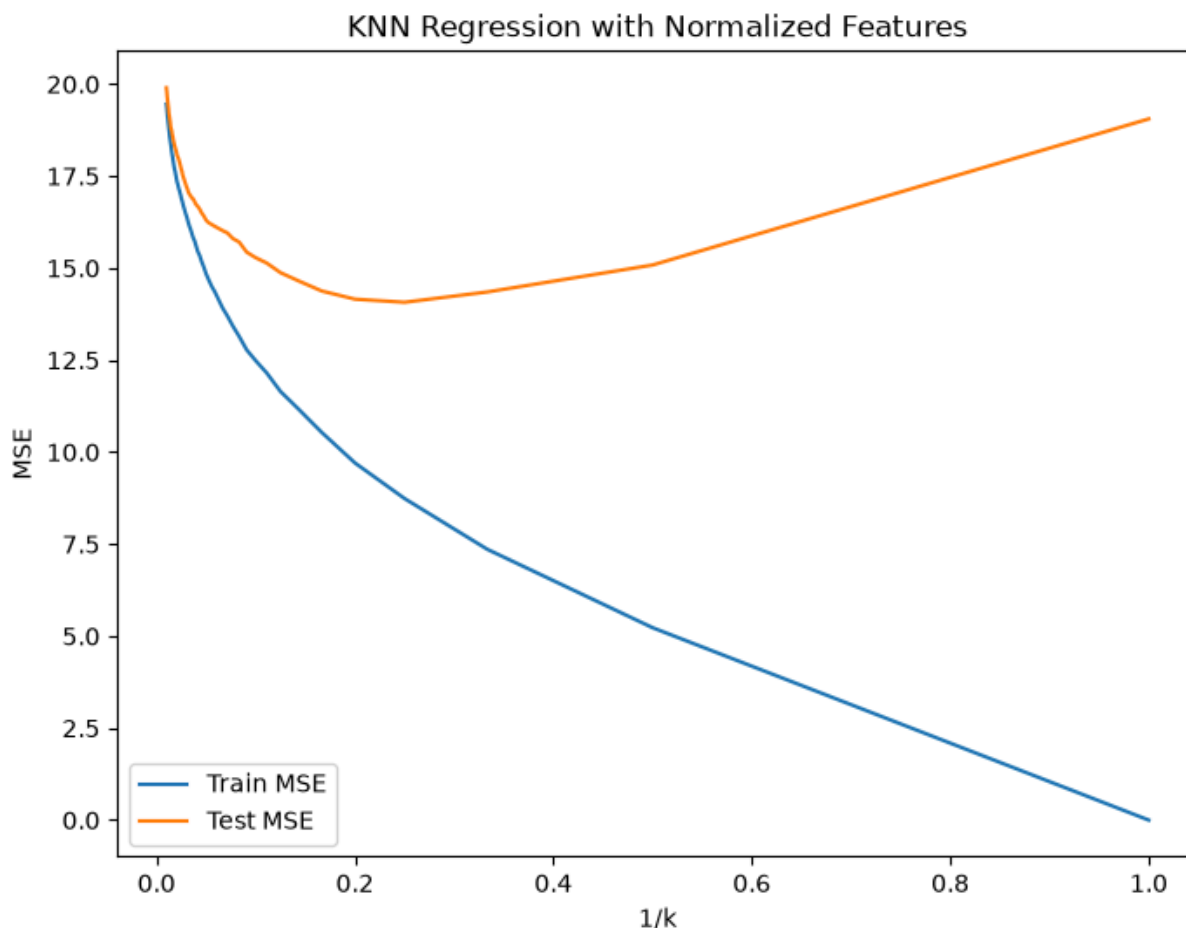
```
In [36]: # Train and test MSE for normalized features versus 1/k

plt.figure(figsize=(8, 6))

plt.plot(inverse_k, normalized_train_mse, label='Train MSE')
plt.plot(inverse_k, normalized_test_mse, label='Test MSE')

plt.xlabel('1/k')
plt.ylabel('MSE')
plt.title('KNN Regression with Normalized Features')
plt.legend()

plt.show()
```



Results:

For KNN regression using the raw features, the best value of k is 5, with a test MSE of approximately 15.7048.

For KNN regression using normalized features, the best value of k is 4, with a lower test MSE of approximately 14.0706.

Therefore, normalization improves the performance of KNN regression on this dataset. This is reasonable because KNN is based on distances, and the predictor variables are measured on different scales.

The plots also show that very small values of k can lead to overfitting, while very large values of k increase the test error.

(j) Compare KNN and Linear

```
In [37]: # Compare the best regression model with the best KNN model
```

```
final_comparison = pd.DataFrame({
    'Model': [
        'Improved Regression',
        'KNN with Normalized Features'
    ]
})
```

```

    ],
    'Best k': [
        np.nan,
        best_normalized_k
    ],
    'Test MSE': [
        reduced_test_mse,
        best_normalized_test_mse
    ]
})

final_comparison

```

Out[37]:

	Model	Best k	Test MSE
0	Improved Regression	NaN	18.263588
1	KNN with Normalized Features	4.0	14.070606

Comparison:

The improved regression model has a test MSE of approximately 18.2636, while the best KNN model using normalized features with $k = 4$ has a lower test MSE of approximately 14.0706.

Therefore, KNN regression provides better predictive performance on the test set for this dataset.

The improved regression model captures nonlinearities and interactions explicitly through polynomial and interaction terms, while KNN can capture more flexible local relationships without assuming a particular functional form. The lower test MSE of KNN suggests that these local nonlinear patterns are useful for predicting PE.

Overall, among the models tested, the normalized KNN model with $k = 4$ gives the lowest test error.

2. ISLR: 2.4.1

For each of parts (a) through (d), indicate whether we would generally expect the performance of a flexible statistical learning method to be better or worse than an inflexible method. Justify your answer.

(a) The sample size n is extremely large, and the number of predictors p is small.

A flexible statistical learning method would generally perform **better**.

Since the sample size is extremely large and the number of predictors is small, there is enough data to estimate a flexible model reliably. The large sample size reduces the risk of overfitting, while the flexible method can capture more complex relationships in the data.

(b) The number of predictors p is extremely large, and the number of observations n is small.

A flexible method would generally perform **worse**.

When the number of predictors is very large but the number of observations is small, a flexible method is more likely to overfit the training data. An inflexible method would have lower variance and may generalize better to new observations.

(c) The relationship between the predictors and response is highly non-linear.

A flexible method would generally perform **better**.

If the relationship between the predictors and the response is highly nonlinear, a flexible method can adapt to the nonlinear pattern. An inflexible method may be too restrictive and have high bias.

(d) The variance of the error terms, i.e. $\sigma^2 = \text{Var}(\epsilon)$, is extremely high.

A flexible method would generally perform **worse**.

When the variance of the error terms is extremely high, much of the variation in the response is due to noise. A flexible method may fit this noise rather than the true underlying relationship, which can lead to overfitting and high variance. An inflexible method is generally more stable in this situation.

3. ISLR: 2.4.7

(a) Compute the Euclidean distance between each observation and the test point, $X_1 = X_2 = X_3 = 0$.

The test point is:

$$(0, 0, 0)$$

Using Euclidean distance,

$$d = \sqrt{(X_1 - 0)^2 + (X_2 - 0)^2 + (X_3 - 0)^2}$$

The distances are:

Observation	Distance
1	$\sqrt{0^2 + 3^2 + 0^2}$ $= 3$
2	$\sqrt{2^2 + 0^2 + 0^2}$ $= 2$
3	$\sqrt{0^2 + 1^2 + 3^2}$ $= \sqrt{10}$
4	$\sqrt{0^2 + 1^2 + 2^2}$ $= \sqrt{5}$
5	$\sqrt{(-1)^2 + 0^2 + 1^2}$ $= \sqrt{2}$
6	$\sqrt{1^2 + 1^2 + 1^2}$ $= \sqrt{3}$

Therefore, ordered from closest to farthest:

5, 6, 2, 4, 1, 3

(b) What is our prediction with $K = 1$? Why?

For $K = 1$, the nearest observation to the test point $(0, 0, 0)$ is Observation 5.

Observation 5 has response **Green**.

Therefore, the predicted class is:

Green

(c) What is our prediction with $K = 3$? Why?

For $K = 3$, the three nearest observations to the test point $(0, 0, 0)$ are:

- Observation 5: **Green**
- Observation 6: **Red**
- Observation 2: **Red**

Since two of the three nearest neighbors are Red, the majority vote is Red.

Therefore, the predicted class is:

Red

(d) If the Bayes decision boundary in this problem is highly non-linear, then would we expect the best value for K to be large or small? Why?

We would expect the best value of K to be relatively **small**.

A small value of K produces a more flexible decision boundary, which is better able to capture a highly nonlinear Bayes decision boundary.

In contrast, a large value of K produces a smoother and less flexible decision boundary, which may fail to capture the nonlinear pattern.